

14 - 面向对象编程 IV

继承与多态

知识点

- 继承
- IS-A vs HAS-A
- 传递性
- 抽象类
- 抽象方法
- 多态
- 方法的关键
- Overriding vs Overloading
- 方法调用的字节
- invokespecial
- invokevirtual (overriding 和 overloading的不同实现)
- 继承中的成员变量

Outline

- 理解继承为什么出现
- 理解 IS-A 与 HAS-A
- 掌握 extends 的使用
- 理解抽象类与抽象方法
- 理解多态的本质
- 掌握 overriding 与 overloading
- 理解动态绑定
- 理解成员变量与方法的区别

为什么需要继承

- 没有继承
 - Student、Teacher、Doctor都有name, birthday, eat()
- 问题：重复代码太多

继承

- Person: 父类 / 超类 / 基类
- Student: 子类
- extends: 继承

```
class Person {  
    String name;  
    String birthday;  
  
    void eat(){  
        System.out.println("eat");  
    }  
}  
  
class Student extends Person {  
}
```

关于继承

- 子类继承了父类所有的成员变量和成员方法
- 可以增加成员变量和成员方法
- 可以覆盖父类的成员方法
- 不可以覆盖父类的成员变量

关于继承

子类继承了父类所有成员变量，包括：

- public
- protected
- default
- private

```
class Father{  
    private int money = 100;  
}
```

```
class Son extends Father{  
}  
Son s = new Son();
```



```
class Son extends Father{  
    void f(){  
        System.out.println(money);  
        // 编译错误  
    }  
}
```

关于继承

```
class Father{  
    private int money = 100;  
  
    public void showMoney(){  
        System.out.println(money);  
    }  
}
```

```
class Son extends Father{  
}  
  
Son s = new Son();  
s.showMoney();
```

关于继承

子类继承父类的方法，包括：

- public方法
- protected方法
- 默认方法

private方法比较特殊。

```
class Father{  
    private void test(){  
        System.out.println("father");  
    }  
}
```

```
class Son extends Father{  
    public void test(){  
        System.out.println("son");  
    }  
}
```

关于继承

子类可以增加成员变量

子类可以增加成员方法

```
class Student extends  
Person{  
  
    int studentId;  
}
```

```
class Student extends Person{  
    void study(){  
  
        System.out.println("study");  
    }  
}
```

关于继承

子类可以覆盖父类成员方法 (Override)

```
class Animal{  
  
    void eat(){  
        System.out.println("animal eat");  
    }  
}
```

```
class Dog extends Animal{  
    @Override  
    void eat(){  
        System.out.println("dog eat bone");  
    }  
}
```

```
Animal a = new Dog();  
a.eat();
```

输出：
dog eat bone

关于继承

成员变量没有Override

```
class A{  
    public String name = "Father";  
}  
  
class B extends A{  
    public String name = "Son";  
}  
  
A a = new B();  
System.out.println(a.name); // "Father"
```

关于继承

特性	方法	成员变量
是否支持多态	支持	不支持
是否动态绑定	是	否
是否能override	能	不能
看什么类型	实际对象类型	引用变量类型

继承的传递

A是一种B、 B是一种C;
A也是一种C

```
class Animal{
    void eat(){
        System.out.println("eat");
    }
}

class Canine extends Animal{
}

class Dog extends Canine{
}

Dog d = new Dog();
d.eat();
```

IS-A

- Student is a Person
- Dog is an Animal
- Cat is an Animal
- 口诀： “是一种” → 可以继承

HAS-A

- Car has an Engine
- Classroom has Students
- Computer has CPU
- HAS-A: 组合关系不是继承关系

组合优于继承

原因：继承耦合太强

```
class Bird{  
    void fly(){}  
}  
  
class Penguin extends Bird{  
}
```

```
class FlyBehavior{  
    void fly(){}  
}  
  
class Bird{  
    FlyBehavior flyBehavior;  
}
```

is-a vs is-like-a

- Is-a
- Dog is an Animal, 适合继承
- is-like-a
- Airplane is like a Bird, 适合组合 / 接口

继承的好处

- 减少重复代码
- 提高复用性
- 统一行为规范父类（定义了一个所有子类必须遵循的契约）
- 更容易扩展

动物园程序

- 每一种动物用一个类表示
- 希望增加新的动物

动物园程序

- 版本一：不使用继承

```
class Tiger {  
    private String name;  
    public void eat(){  
        System.out.println(name + "吃肉");  
    }  
    public void shout(){  
        System.out.println(name + "嗷嗷叫");  
    }  
}
```

Tiger tiger = new Tiger("小虎",4);

```
class Monkey {  
    private String name;  
    public void eat(){  
        System.out.println(name + "吃香蕉");  
    }  
    public void shout(){  
        System.out.println(name + "吱吱叫");  
    }  
}
```

Monkey monkey = new Monkey("小猴",2);

- 缺点：属性、构造器大量重复；新增动物需完整编写全套代码，维护繁琐

动物园程序

- 版本二：单继承，抽取公共父类 Animal，子类复用通用代码，仅重写特有行为

```
class Tiger extends Animal{
    @Override
    public void eat() {
        System.out.println(name + "吃肉");
    }
    @Override
    public void shout() {
        System.out.println(name + "嗷嗷叫");
    }
}
```

```
class Monkey extends Animal{
    @Override
    public void eat() {
        System.out.println(name + "吃香蕉");
    }
    @Override
    public void shout() {
        System.out.println(name + "吱吱叫");
    }
}
```

```
Animal tiger = new Tiger("猛虎",5);
```

- 优势：公共代码只写一次；新增动物仅创建子类，重写独有方法即可

动物园程序

- 版本三：多级继承，Animal 动物 → Mammal 哺乳动物 → 老虎 / 狮子

```
class Mammal extends Animal {  
    public Mammal(String name, int age) {  
        super(name, age);  
    }  
    // 哺乳动物特有行为  
    public void born() {  
        System.out.println(name + "胎生繁衍  
");  
    }  
    public void eat(){}  
}
```

```
// 三级子类：老虎，继承哺乳动物  
class Tiger extends Mammal {  
    public Tiger(String name, int age) {  
        super(name, age);  
    }  
    @Override  
    public void eat() {  
        System.out.println(name + "捕食肉类  
");  
    }  
}
```

- 所有动物共有的：name、age、breath() → 写在 Animal
- 所有哺乳动物共有的：born()（胎生）、milk()（哺乳） → 写在 Mammal
- 每种动物自己的：eat() → 写在 Tiger/Lion
- 优势：代码复用最大化；结构更清晰，符合现实世界逻辑

动物园程序

- 新增物种：熊猫

```
class Panda extends Mammal {  
    @Override  
    public void eat() {  
        System.out.println("吃竹子");  
    }  
}
```

- 不用写 name/age
- 不用写 breath ()
- 不用写 胎生 / 哺乳
- 原有代码完全不用改
- 优势：符合**开闭原则**，对扩展开放，对修改关闭。

抽象类和抽象方法

是否有动物对象？

- `Wolf aWolf = new Wolf();`
- `Animal aHippo = new Hippo();`
- 很奇怪: `Animal animal = new Animal();`

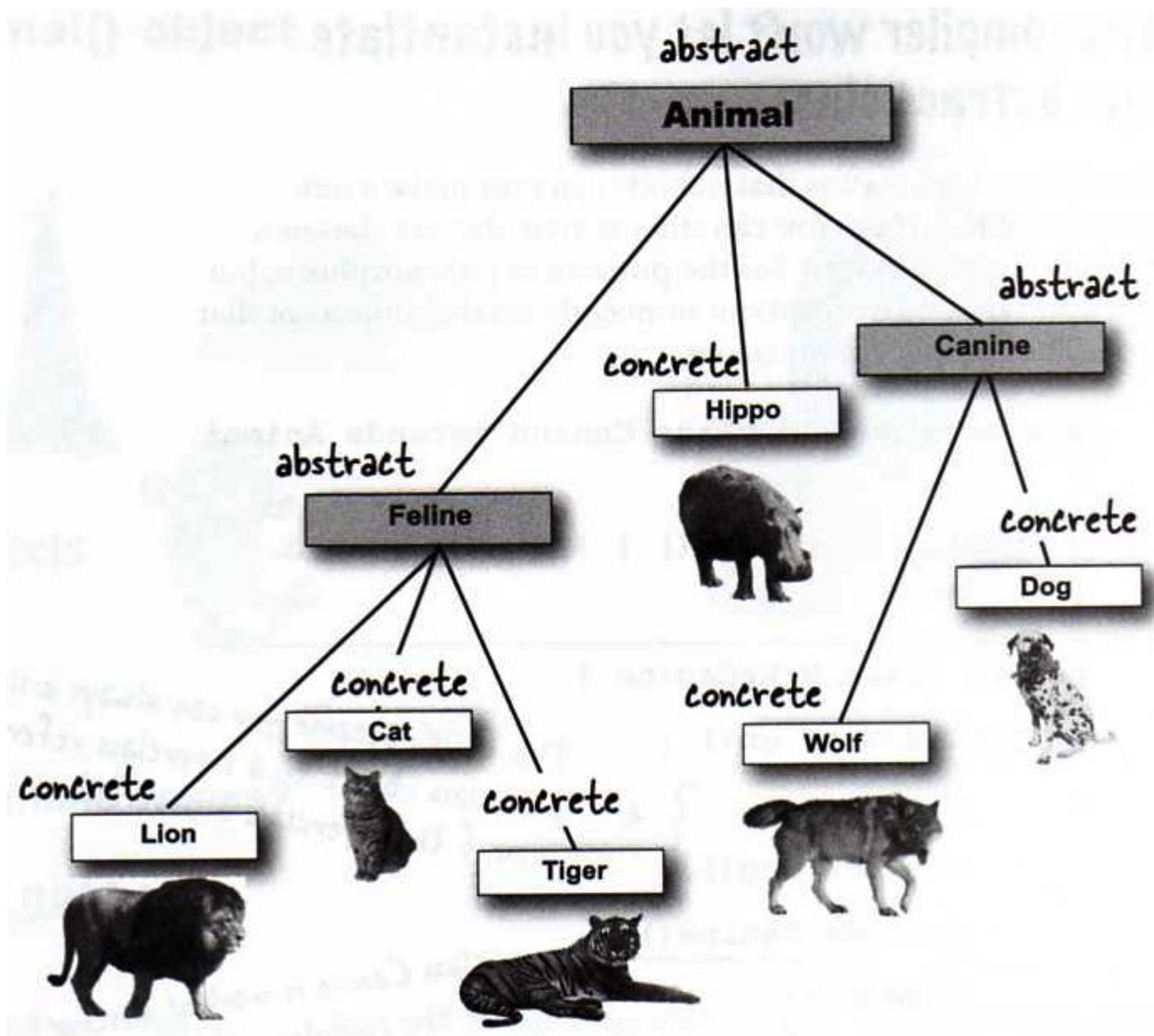
一些类不应该实例化

```
abstract class Animal{ }
```

```
Animal animal = new Animal() // 编译错误
```

抽象类本身没有用（除非他被继承）
抽象类的子类可以实例化

新的继承树



抽象方法

- 没有实现主体的方法
 - `public abstract void eat();`

绕口令

- 非抽象类中不可以有抽象方法
- 抽象类中可以有非抽象方法

```
// 抽象类
abstract class Animal {
    // 抽象方法 （没有方法体）
    public abstract void eat();

    // 普通成员方法 （有具体实现）
    public void sleep() {
        System.out.println(name + " 正在睡觉");
    }
}

// 子类必须重写抽象方法
class Dog extends Animal {

    @Override
    public void eat() {
        System.out.println(name + " 吃骨头");
    }
}
```

实现抽象方法

- 抽象方法的存在旨在实现多态特性。
- 子类必须实现父类全部抽象方法。
- 抽象方法的实现过程等同于重写父类方法。

```
abstract class Animal{  
    public abstract void makeNoise();  
}
```

Dog -> 汪汪

Cat -> 喵喵

Lion -> 吼叫

```
animal.makeNoise();
```

多态

只能放dog的list

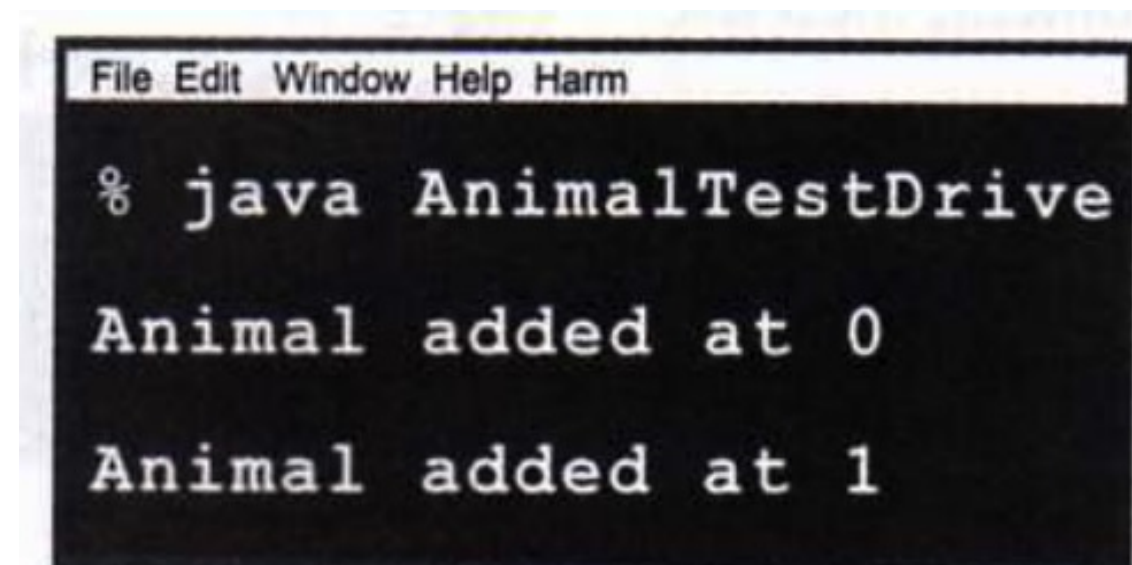
```
public class MyDogList{  
    private Dog[] dogs = new Dog[5];  
    private int nextIndex = 0;  
  
    public void add(Dog d){  
        if(nextIndex < dogs.length){  
            dogs[nextIndex]= d;  
            System.out.println( "Dog added at " + nextIndex);  
            nextIndex++;  
        }  
    }  
}
```

可以放任何动物的list

```
public class MyAnimalList{
    private Animal[] animals = new
Animal[5];
    private int nextIndex = 0;

    public void add(Animal a){
        if(nextIndex < animals.length){
            animals[nextIndex]= a;
            System.out.println( "Animal
added at " + nextIndex);
            nextIndex++;
        }
    }
}
```

```
public class AnimalTestDrive{
    public static void main(String[] args)
        MyAnimalList list = new
MyAnimalList();
        Dog d = new Dog();
        Cat c = new Cat();
        list.add(d);
        list.add(c);
    }
}
```



The screenshot shows a terminal window with a menu bar (File, Edit, Window, Help, Harm) and the following output:

```
% java AnimalTestDrive
Animal added at 0
Animal added at 1
```

list中那可不可以放不是Animal的类?

再进一步，有没有可以放任何类的list呢？

如果我们可以创建一个所有类的
父类就可以！

Java.lang.Object

Object类的方法

- equals(Object o)
 - 指示某个其他对象是否与此对象 “相等” 。
- getClass()
 - 返回一个对象的运行时类。
- hashCode()
 - 返回该对象的哈希码值。
- toString()
 - 返回该对象的字符串表示。

多态的代价

- 使用Dog List

```
ArrayList<Dog> myDogArrayList = new ArrayList<Dog>();
```

```
Dog aDog = new Dog();
```

```
myDogArrayList.add(aDog);
```

```
Dog d = myDogArrayList.get(0);
```

多态的代价

- 使用Object List

```
ArrayList<Object> myDogArrayList = new ArrayList<Object>();
```

```
Dog aDog = new Dog();
```

```
myDogArrayList.add(aDog);
```

```
Dog d = myDogArrayList.get(0); // 编译报错
```

编译器的眼里

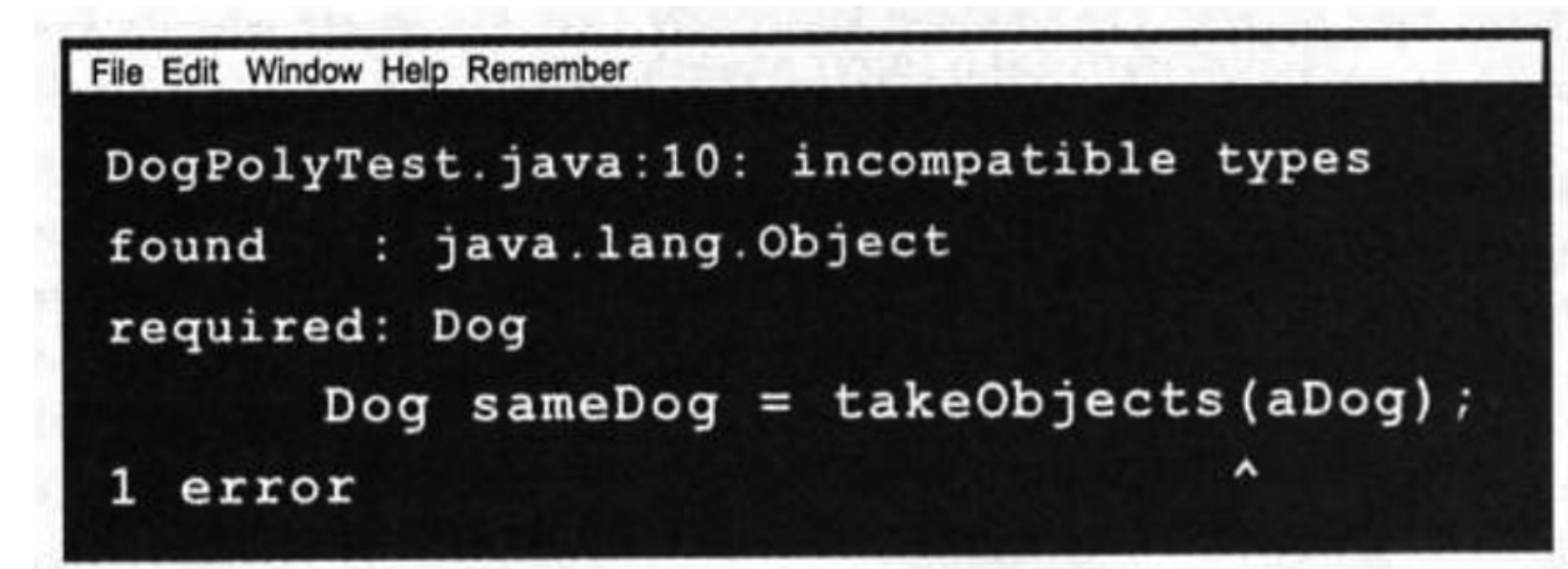
只看引用变量类型
不看实际对象类型

```
Object o = new Dog();
```

编译器只认为：o是Object。

取出Object类型的变量无法赋给Dog类型的变量

```
public void go () {  
  
    Dog d = newDog();  
  
    Dog sameDog = takeObject(d);  
  
}  
  
public Object takeObject(Object o){  
  
    return o;  
  
}
```

A screenshot of a Java IDE window showing a compilation error. The window has a menu bar with 'File', 'Edit', 'Window', 'Help', and 'Remember'. The main text area displays the following error message: 'DogPolyTest.java:10: incompatible types found : java.lang.Object required: Dog'. Below this, the line of code is shown: 'Dog sameDog = takeObjects(aDog);'. An arrow points to the closing parenthesis of 'takeObjects(aDog);'. At the bottom, it says '1 error'.

取出Object类型的变量可以赋给Object类型的变量

```
public void go () {  
    Dog d = newDog();  
    Object sameDog = takeObject(d);  
}  
  
public Object takeObject(Object o){  
    return o;  
}
```

但是。。。

无法调用Dog的方法

sameDog.bark()

只能调用Object的方法

sameDog.hashCode()



强制类型转换

```
Dog d = (Dog) sameDog;  
d.bark();
```

两句关键

- 编译时，编译器决定是否调用某个方法
依据引用变量的类型，而不是引用变量指向的对象的类型
- 执行时，JVM虚拟机决定实际哪个方法被调用
依据实际引用变量指向的对象的类型

多态

一个对象可以被当作：

- 自己类型
- 父类类型
- Object类型

```
Dog d = new Dog();  
Animal a = d;  
Object o = d;
```

多态的理解

- 一个对象包含了从每一个超类（父类）中继承的所有东西
- 也意味着每一个对象实际也是其超类的类型的对象
- 每个对象都是Object类的对象

强制类型转换的风险

```
Dog d = (Dog) o;  
向下转型有风险
```

```
Object o = new Cat();  
Dog d = (Dog)o;  
运行时: ClassCastException
```

契约

- Dog类的契约应该包括
 - Object类的契约
 - Animal类的契约
 - Canine类的契约
 - Dog类的契约

里氏替换原则

多态的思想

- 多态通过分离“做什么”和“怎么做”，从另一角度将接口和实现分离开来。
- 而多态的作用则是消除类型之间的耦合关系多态方法调用允许一种类型表现出与其他相似类型之间的区别，只要它们都是从同一基类导出而来的。这种区别是根据方法行为的不同来表示出来的，虽然这些方法都可以通过同一个基类来调用。
- 多态可以表达不同的计算类型，并且在运行的时候动态的确定正确的计算。
- 多态是指多个方法使用同一个名字有多种解释，当使用这个名字去调用方法时，系统将选择重载自动的选择其中的一个方法。在多态中只关心一个对象做什么，而不关心如何去做。

同一种类型

```
Dog myDog = new Dog();
```

不同一种类型

```
Animal myDog = new Dog();
```



引用变量的类型是实际对象类型的超类。

多态的形式 — 引用变量

```
Animal [] animals = new Animal[5];
```

```
animals[0] = new Dog();
```

```
animals[1] = new Cat();
```

```
...
```

```
for (int i=0; i< animals.length; i++){
```

```
    animals[i].eat();
```

```
    animals[i].roam();
```

```
}
```

多态的形式 — 参数

```
class Vet{  
    public void giveShot(Animal a){  
        a.makeNoise();  
    }  
}
```

```
class PetOwner{  
    public void start(){  
        Vet v = new Vet();  
        Dog d = new Dog();  
        Hippo h = new Hippo();  
        v.giveShot(d);  
        v.giveShot(h);  
    }  
}
```

多态的形式 — 返回值

```
public Animal getPet(int type){  
    if(type==1) return new Cat();  
    else return new Dog();  
}
```

假如要引入新的动物

- 只需要继承Animal类;
- 通过多态, 就不需要修改原有的表达“做什么”的代码;
- 新动物特有的“怎么做”的行为通过方法覆盖Overriding来动态绑定。

Overriding重写的规则

- 条件
 - 在不同类（父类和子类）、同一个方法、方法名字相同
- 参数必须一致
 - 个数、类型、顺序全部一致
- 返回值必须兼容
 - 例如：父类返回Animal，子类返回dog
- 方法的可达性（权限）不能降低
 - 例如：父类public，子类不能protected

Overloading重载的规则

- 条件：
 - 同一个类中，不同方法，恰好名字相同
- 参数不同（类型、个数、顺序）
- 返回值可以不同，但是不能知识返回值不同
- 方法的可达性提高降低都行

陷阱：“overriding” 私有方法

```
class Derived extends PrivateOverride {  
    public void f() {  
        System.out.println("public f()");  
    }  
}  
  
public class PrivateOverride {  
    private static Test monitor = new Test();  
    private void f() {  
        System.out.println("private f()");  
    }  
    public static void main(String[] args) {  
        PrivateOverride po = new Derived();  
        po.f();  
    }  
}
```

子类不能重写父类的私有方法
编译时已经静态绑定

```
class Base {  
    public void callFoo() {  
        foo();  
    }  
    private void foo() {  
    }  
}  
class Child extends Base {  
    private void foo() {  
  
    }  
}
```

```
Child c = new Child();  
c.callFoo();
```

这个呢？

方法调用的字节码

方法的调用

Java虚拟机里面提供四种方法调用字节码指令

- `invokestatic`: 静态方法
- `invokespecial`: 实例构造器<init>方法, 私有方法和父类方法
- `invokevirtual`: 虚方法
- `invokeinterface`: 接口方法, 会在运行时再确定一个实现此接口的对象
- `invokedynamic`: 先在运行时动态解析出调用点限定符所引用的方法, 然后再执行该方法。把如何查找目标方法的决定权从虚拟机转嫁到具体用户代码之中, 让用户 (包含其他语言的设计者) 有更高的自由度。

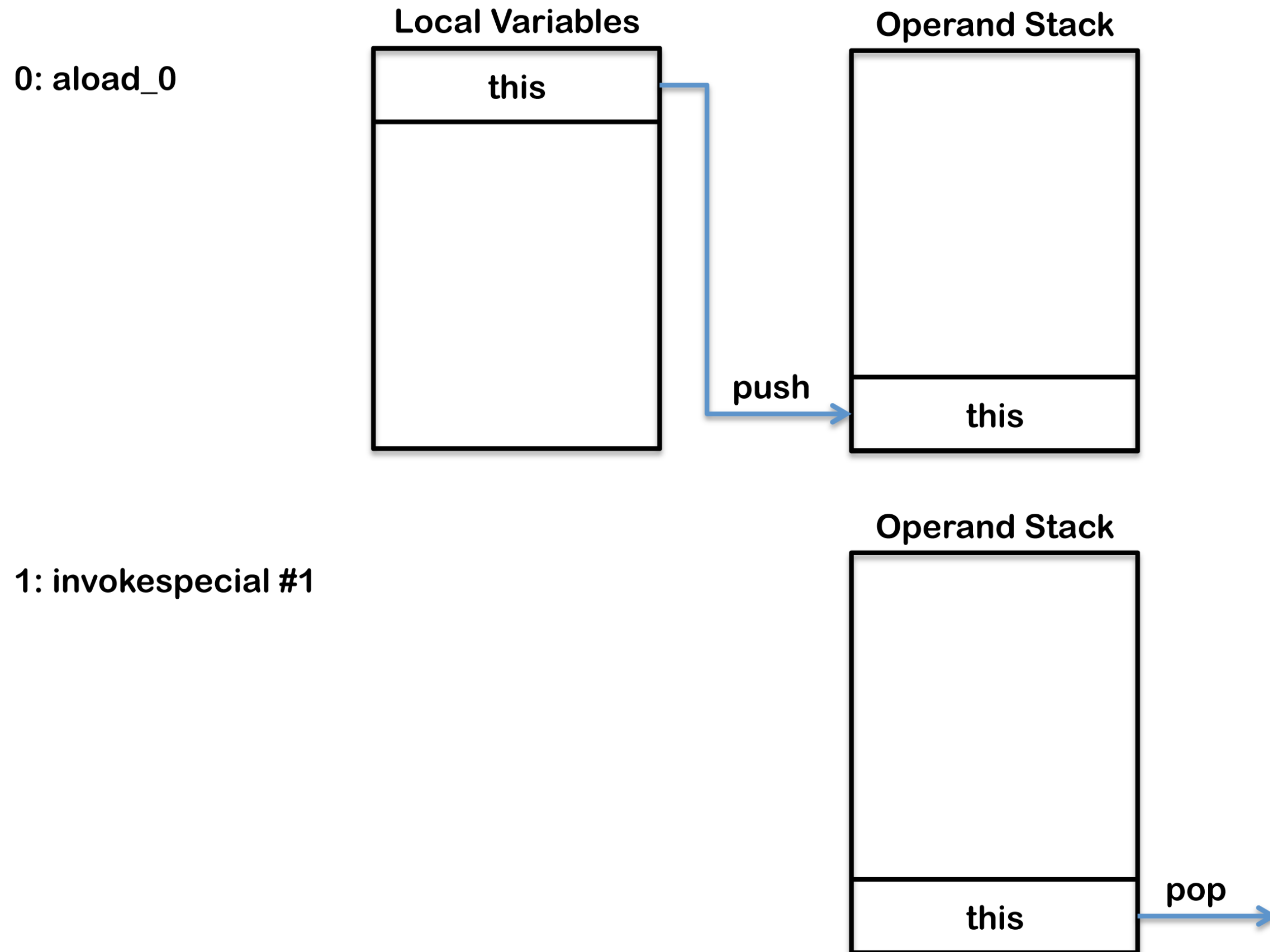
实例文件

```
public class TestClass{  
    private int m;  
  
    public int inc(){  
        return m+1;  
    }  
}
```

```
promote:~ qinliu$ javac TestClass.java
promote:~ qinliu$ javap -verbose TestClass
警告: 二进制文件TestClass包含
org.fenixsoft.clazz.TestClass
Classfile /Users/qinliu/TestClass.class
  Last modified 2015-4-15; size 295 bytes
  MD5 checksum 81f2ab948a7a3068839b61a8f91f634b
  Compiled from "TestClass.java"
public class org.fenixsoft.clazz.TestClass
  minor version: 0
  major version: 52
  flags: ACC_PUBLIC, ACC_SUPER
Constant pool:
  #1 = Methodref          #4.#15          //
java/lang/Object."<init>":()V
  #2 = Fieldref           #3.#16          //
org/fenixsoft/clazz/TestClass.m:I
  #3 = Class              #17            //
org/fenixsoft/clazz/TestClass
  #4 = Class              #18            //
java/lang/Object
  #5 = Utf8               m
  #6 = Utf8               I
  #7 = Utf8               <init>
  #8 = Utf8               ()V
  #9 = Utf8               Code
#10 = Utf8               LineNumberTable
#11 = Utf8               inc
#12 = Utf8               ()I
#13 = Utf8               SourceFile
#14 = Utf8               TestClass.java
#15 = NameAndType        #7:#8          //
  #16 = NameAndType        #5:#6          //
  #17 = Utf8
org/fenixsoft/clazz/TestClass
  #18 = Utf8               java/lang/Object
{
  public org.fenixsoft.clazz.TestClass();
    descriptor: ()V
    flags: ACC_PUBLIC
    Code:
      stack=1, locals=1, args_size=1
        0: aload_0
        1: invokespecial #1          //
Method java/lang/Object."<init>":()V
      4: return
    LineNumberTable:
      line 3: 0

  public int inc();
    descriptor: ()I
    flags: ACC_PUBLIC
    Code:
      stack=2, locals=1, args_size=1
        0: aload_0
        1: getfield      #2          //
Field m:I
        4: iconst_1
        5: iadd
        6: ireturn
    LineNumberTable:
      line 8: 0
}
SourceFile: "TestClass.java"
```

SimpleClass()



invokespecial

```
public class SimpleClass {  
    public void sayHello() {  
        System.out.println("Hello");  
    }  
}
```

```
javap -v -p -s -sysinfo -constants classes/org/jvminternals/SimpleClass.class  
public class org.jvminternals.SimpleClass  
    SourceFile: "SimpleClass.java"  
    minor version: 0  
    major version: 51  
    flags: ACC_PUBLIC, ACC_SUPER  
Constant pool:  
    #1 = Methodref          #6.#17          // java/lang/Object."<init>":()V  
    #2 = Fieldref           #18.#19          // java/lang/System.out:Ljava/io/PrintStream;  
    #3 = String              #20              // "Hello"  
    #4 = Methodref          #21.#22          // java/io/PrintStream.println:(Ljava/lang/String;)V  
    #5 = Class                #23              // org/jvminternals/SimpleClass  
    #6 = Class                #24              // java/lang/Object  
    #7 = Utf8                <init>  
    #8 = Utf8                ()V  
    #9 = Utf8                Code  
    #10 = Utf8              LineNumberTable  
    #11 = Utf8               LocalVariableTable  
    #12 = Utf8               this  
    #13 = Utf8               Lorg/jvminternals/SimpleClass;  
    #14 = Utf8               sayHello  
    #15 = Utf8               SourceFile  
    #16 = Utf8               SimpleClass.java  
    #17 = NameAndType         #7:#8          // "<init>":()V  
    #18 = Class                #25              // java/lang/System  
    #19 = NameAndType         #26:#27         // out:Ljava/io/PrintStream;  
    #20 = Utf8                Hello  
    #21 = Class                #28              // java/io/PrintStream  
    #22 = NameAndType         #29:#30         // println:(Ljava/lang/String;)V  
    #23 = Utf8                org/jvminternals/SimpleClass  
    #24 = Utf8                java/lang/Object  
    #25 = Utf8                java/lang/System  
    #26 = Utf8                out  
    #27 = Utf8                Ljava/io/PrintStream;  
    #28 = Utf8                java/io/PrintStream  
    #29 = Utf8                println  
    #30 = Utf8                (Ljava/lang/String;)V  
}
```

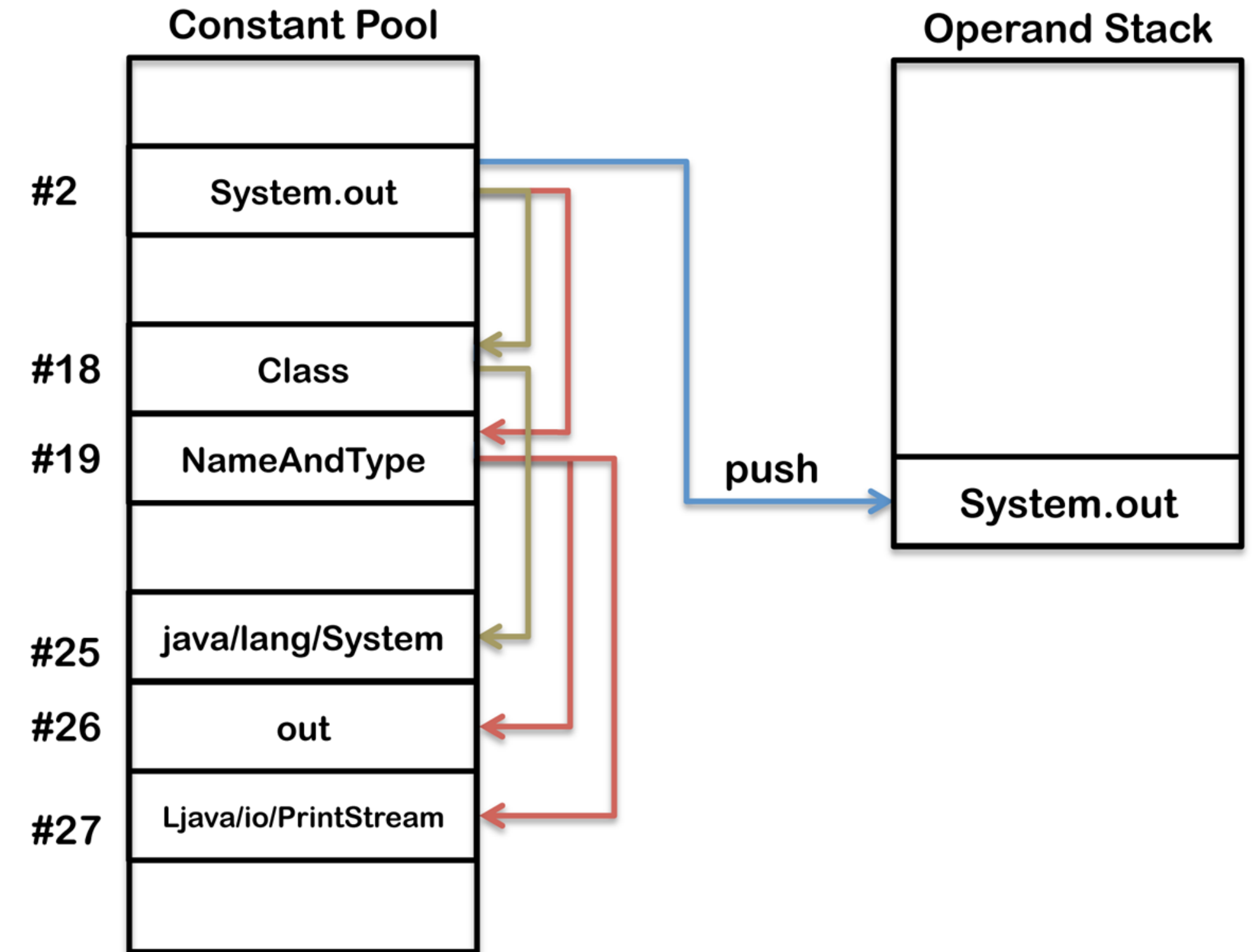
```
public org.jvminternals.SimpleClass();
Signature: ()V
flags: ACC_PUBLIC
Code:
    stack=1, locals=1, args_size=1
        0: aload_0
        1: invokespecial #1    // Method java/lang/Object."<init>":()V
        4: return
LineNumberTable:
    line 3: 0
LocalVariableTable:
    Start   Length  Slot  Name   Signature
         0         5      0   this   Lorg/jvminternals/SimpleClass;
```

```
public void sayHello();
Signature: ()V
flags: ACC_PUBLIC
Code:
    stack=2, locals=1, args_size=1
        0: getstatic      #2    // Field java/lang/System.out:Ljava/io/PrintStream;
        3: ldc            #3    // String "Hello"
        5: invokevirtual  #4    // Method java/io/PrintStream.println:(Ljava/lang/String;)V
        8: return
LineNumberTable:
    line 6: 0
    line 7: 8
LocalVariableTable:
    Start   Length  Slot  Name   Signature
```

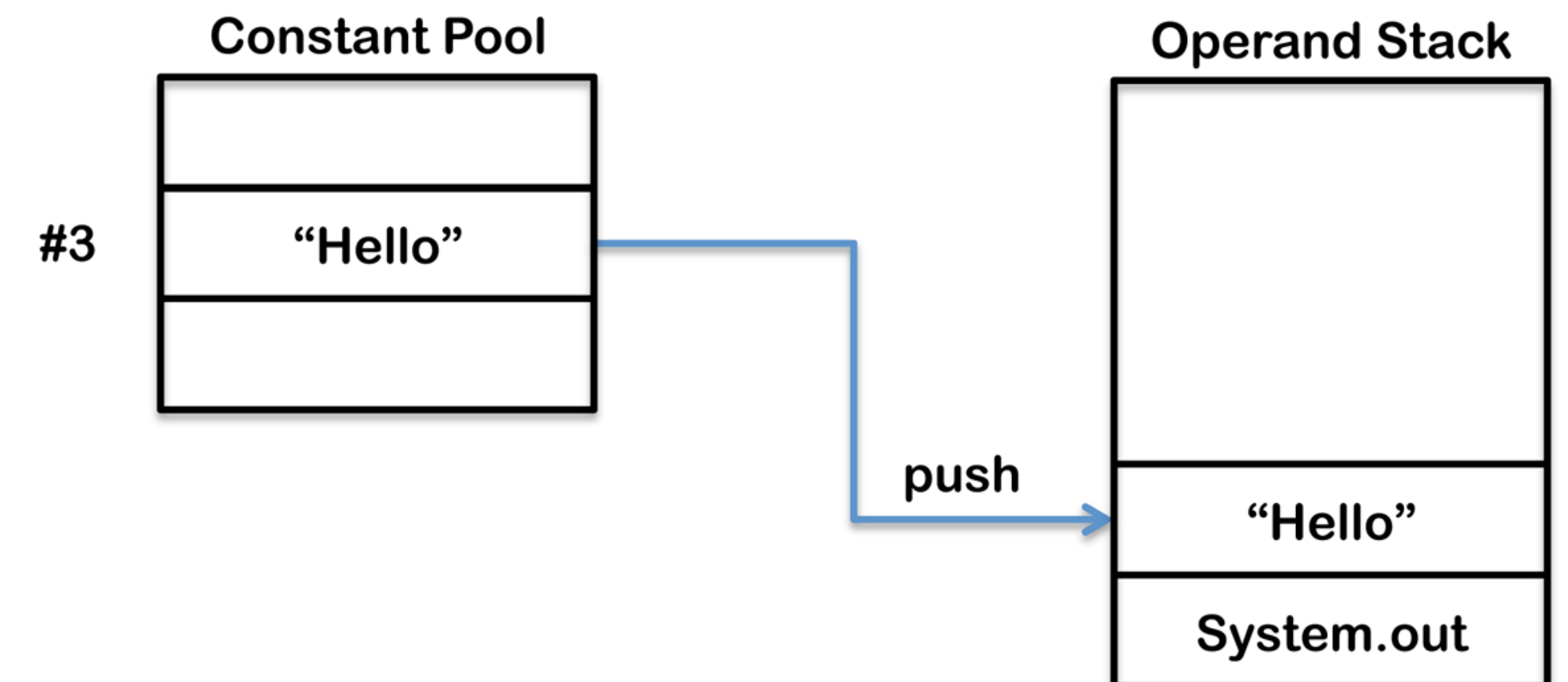
invokevirtual

sayHello()

0: getstatic



3: ldc



invokevirtual

3: ldc

#3

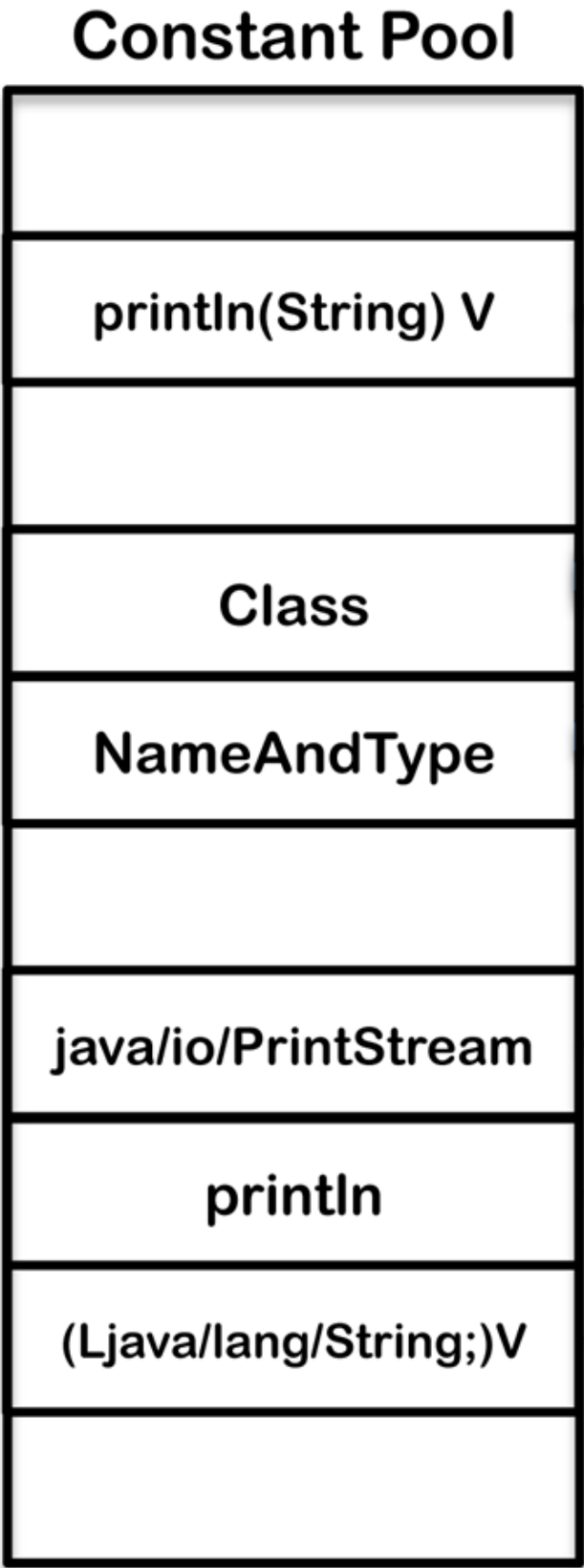


push



5: invokevirtual #4

#4



pop

pop

分派与绑定（重载、重写的本质根源）

- 调用一个对象的方法时：调用对象方法 = 先分派 + 后绑定

分派(dispatch)，即确定要调用的具体方法。筛选匹配出的目标方法。

绑定(binding)，将方法调用与实际执行的代码（方法体）关联起来。

- 编译期机制：静态绑定 + 多分派（对应 Overload 重载）

静态绑定（在编译时确定方法调用与实际执行的代码之间的关系）

多分派（根据调用对象和参数类型来确定要执行的方法）

overloading = 编译期 + 静态绑定 + 多分派

- 运行期

动态绑定（在运行时确定方法调用与实际执行的代码之间的关系）

单分派（根据方法调用对象来确定要执行的方法）

overriding = 运行期 + 动态绑定 + 单分派

- 都是invokevirtual指令

```
public class Overload {  
  
    public static void sayHello(Object arg) {  
        System.out.println("hello Object");  
    }  
    public static void sayHello(int arg) {  
        System.out.println("hello int");  
    }  
    public static void sayHello(long arg) {  
        System.out.println("hello long");  
    }  
    public static void sayHello(Character arg) {  
        System.out.println("hello Character");  
    }  
    public static void sayHello(char arg) {  
        System.out.println("hello char");  
    }  
    public static void sayHello(char... arg) {  
        System.out.println("hello char ...");  
    }  
    public static void sayHello(Serializable arg) {  
        System.out.println("hello Serializable");  
    }  
    public static void main(String[] args) {  
        sayHello('a');  
    }  
}
```

静态分派

overloading匹配优先级

- 编译器可知
- `sayHello( 'a' )`
- `char > int > long > Character > Serializable > Object > char...`

- 上面的代码运行后会输出: hello char
这很好理解, ' a ' 是一个char类型的数据, 自然会寻找参数类型为char的重载方法
- 注释掉 sayHello(char arg) 方法, 输出: hello int
这时发生了一次自动类型转换, ' a ' 除了可以代表一个字符串, 还可以代表数字97。因此参数类型为int的重载也是合适的。
- 继续注释掉 sayHello(int arg)方法, 输出: hello long
这时发生了两次自动类型转换, 'a ' 转型为整数97之后, 进一步转型为长整数97L, 匹配了参数类型为long的重载。
- 继续注释掉 sayHello(long arg)方法, 输出: hello Character
这时发生了一次自动装箱, ' a ' 被包装为它的封装类型 java.lang.Character, 所以匹配到了参数类型为Character的重载。

例子

```
public class SayHello{
    public void sayHello(char c){
        System.out.println("say hello char:" + c);
    }
    public void sayHello(int c){
        System.out.println("say hello int:" + c);
    }
    public void sayHello(long c){
        System.out.println("say hello long:" + c);
    }

    public static void main(String [] args){
        SayHello sh = new SayHello();
        sh.sayHello('a');
    }
}
```

Last modified 2015-5-20; size 995 bytes
MD5 checksum 68d802eacf261f5ee630b3cf04d61967
Compiled from "SayHello.java"

public class SayHello
 minor version: 0
 major version: 52
 flags: ACC_PUBLIC, ACC_SUPER

Constant pool:
 #1 = Methodref #17.#30 // java/lang/Object."<init>":()V
 #2 = Fieldref #31.#32 // java/lang/System.out:Ljava/io/PrintStream;
 #3 = Class #33 // java/lang/StringBuilder
 #4 = Methodref #3.#30 // java/lang/StringBuilder."<init>":()V
 #5 = String #34 // say hello char:
 #6 = Methodref #3.#35 // java/lang/StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;
 #7 = Methodref #3.#36 // java/lang/StringBuilder.append:(C)Ljava/lang/StringBuilder;
 #8 = Methodref #3.#37 // java/lang/StringBuilder.toString:()Ljava/lang/String;
 #9 = Methodref #38.#39 // java/io/PrintStream.println:(Ljava/lang/String;)V
 #10 = String #40 // say hello int:
 #11 = Methodref #3.#41 // java/lang/StringBuilder.append:(I)Ljava/lang/StringBuilder;
 #12 = String #42 // say hello long:
 #13 = Methodref #3.#43 // java/lang/StringBuilder.append:(J)Ljava/lang/StringBuilder;
 #14 = Class #44 // SayHello
 #15 = Methodref #14.#30 // SayHello."<init>":()V
 #16 = Methodref #14.#45 // SayHello.sayHello:(C)V
 #17 = Class #46 // java/lang/Object
 #18 = Utf8 <init>
 #19 = Utf8 ()V
 #20 = Utf8 Code
 #21 = Utf8 LineNumberTable
 #22 = Utf8 sayHello
 #23 = Utf8 (C)V
 #24 = Utf8 (I)V
 #25 = Utf8 (J)V
 #26 = Utf8 main
 #27 = Utf8 ([Ljava/lang/String;)V
 #28 = Utf8 SourceFile
 #29 = Utf8 SayHello.java
 #30 = NameAndType #18:#19 // "<init>":()V
 #31 = Class #47 // java/lang/System
 #32 = NameAndType #48:#49 // out:Ljava/io/PrintStream;
 #33 = Utf8 java/lang/StringBuilder
 #34 = Utf8 say hello char:
 #35 = NameAndType #50:#51 // append:(Ljava/lang/String;)Ljava/lang/StringBuilder;
 #36 = NameAndType #50:#52 // append:(C)Ljava/lang/StringBuilder;
 #37 = NameAndType #53:#54 // toString:()Ljava/lang/String;
 #38 = Class #55 // java/io/PrintStream
 #39 = NameAndType #56:#57 // println:(Ljava/lang/String;)V
 #40 = Utf8 say hello int:
 #41 = NameAndType #50:#58 // append:(I)Ljava/lang/StringBuilder;
 #42 = Utf8 say hello long:
 #43 = NameAndType #50:#59 // append:(J)Ljava/lang/StringBuilder;
 #44 = Utf8 SayHello
 #45 = NameAndType #22:#23 // sayHello:(C)V
 #46 = Utf8 java/lang/Object
 #47 = Utf8 java/lang/System
 #48 = Utf8 out
 #49 = Utf8 Ljava/io/PrintStream;
 #50 = Utf8 append
 #51 = Utf8 (Ljava/lang/String;)Ljava/lang/StringBuilder;
 #52 = Utf8 (C)Ljava/lang/StringBuilder;
 #53 = Utf8 toString
 #54 = Utf8 ()Ljava/lang/String;
 #55 = Utf8 java/io/PrintStream
 #56 = Utf8 println
 #57 = Utf8 (Ljava/lang/String;)V

```

public SayHello();
  descriptor: ()V
  flags: ACC_PUBLIC
  Code:
    stack=1, locals=1, args_size=1
      0: aload_0
      1: invokespecial #1
      4: return
  LineNumberTable:
    line 1: 0

public void sayHello(char);
  descriptor: (C)V
  flags: ACC_PUBLIC
  Code:
    stack=3, locals=2, args_size=2
      0: getstatic      #2
      3: new            #3
      6: dup
      7: invokespecial #4
     10: ldc           #5
     12: invokevirtual #6
     15: iload_1
     16: invokevirtual #7
     19: invokevirtual #8
     22: invokevirtual #9
     25: return
  LineNumberTable:
    line 3: 0
    line 4: 25

public void sayHello(int);
  descriptor: (I)V
  flags: ACC_PUBLIC
  Code:
    stack=3, locals=2, args_size=2
      0: getstatic      #2
      3: new            #3
      6: dup
      7: invokespecial #4
     10: ldc           #10
     12: invokevirtual #6
     15: iload_1
     16: invokevirtual #11
     19: invokevirtual #8
     22: invokevirtual #9
     25: return
  LineNumberTable:
    line 6: 0
    line 7: 25

```

// Method java/lang/Object."<init>":()V

// Field java/lang/System.out:Ljava/io/PrintStream;
// class java/lang/StringBuilder

// Method java/lang/StringBuilder."<init>":()V
// String say hello char:
// Method java/lang/StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;

// Method java/lang/StringBuilder.append:(C)Ljava/lang/StringBuilder;
// Method java/lang/StringBuilder.toString:()Ljava/lang/String;
// Method java/io/PrintStream.println:(Ljava/lang/String;)V

// Field java/lang/System.out:Ljava/io/PrintStream;
// class java/lang/StringBuilder

// Method java/lang/StringBuilder."<init>":()V
// String say hello int:
// Method java/lang/StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;

// Method java/lang/StringBuilder.append:(I)Ljava/lang/StringBuilder;
// Method java/lang/StringBuilder.toString:()Ljava/lang/String;
// Method java/io/PrintStream.println:(Ljava/lang/String;)V

```
descriptor: (J)V
flags: ACC_PUBLIC
Code:
    stack=4, locals=3, args_size=2
        0: getstatic      #2          // Field java/lang/System.out:Ljava/io/PrintStream;
        3: new              #3          // class java/lang/StringBuilder
        6: dup
        7: invokespecial  #4          // Method java/lang/StringBuilder."<init>":()V
       10: ldc              #12         // String say hello long:
       12: invokevirtual #6          // Method java/lang/StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;
       15: lload_1
       16: invokevirtual #13         // Method java/lang/StringBuilder.append:(J)Ljava/lang/StringBuilder;
       19: invokevirtual #8          // Method java/lang/StringBuilder.toString:()Ljava/lang/String;
       22: invokevirtual #9          // Method java/io/PrintStream.println:(Ljava/lang/String;)V
       25: return
LineNumberTable:
    line 9: 0
    line 10: 25

public static void main(java.lang.String[]);
descriptor: ([Ljava/lang/String;)V
flags: ACC_PUBLIC, ACC_STATIC
Code:
    stack=2, locals=2, args_size=1
        0: new              #14          // class SayHello
        3: dup
        4: invokespecial  #15          // Method "<init>":()V
        7: astore_1
        8: aload_1
        9: bipush          97
       11: invokevirtual #16          // Method sayHello:(C)V
       14: return
LineNumberTable:
    line 13: 0
    line 14: 8
    line 15: 14
    }
SourceFile: "SayHello.java"
```

例子

```
public class SayHello{
    public void sayHello(int c){
        System.out.println("say hello
int:" + c);
    }
    public void sayHello(long c){
        System.out.println("say hello
long:" + c);
    }
    public static void main(String [] args){
        SayHello sh = new SayHello();
        sh.sayHello('a');
    }
}
```

```

public static void main(java.lang.String[]);
  descriptor: ([Ljava/lang/String;)V
  flags: ACC_PUBLIC, ACC_STATIC
  Code:
    stack=2, locals=2, args_size=1
       0: new          #12          // class SayHello
       3: dup
       4: invokespecial #13          // Method "<init>":()V
       7: astore_1
       8: aload_1
       9: bipush       97
      11: invokevirtual #14          // Method sayHello:(I)V
      14: return
 LineNumberTable:
    line 11: 0
    line 12: 8
    line 13: 14
}

```

动态分派

```
static abstract class Human {  
    protected abstract void sayHello();  
}  
  
static class Man extends Human {  
    @Override  
    protected void sayHello() {  
        System.out.println("man say hello");  
    }  
}  
  
static class Woman extends Human {  
    @Override  
    protected void sayHello() {  
        System.out.println("woman say  
hello");  
    }  
}
```

```
public static void main(String[] args) {  
    Human man = new Man();  
    Human woman = new Woman();  
    man.sayHello();  
    woman.sayHello();  
    man = new Woman();  
    man.sayHello();  
}
```

```
man say hello  
woman say hello  
woman say hello
```

```

Classfile /Users/liuqin/Downloads/DynamicDispatch.class
promote:Downloads liuqin$ javap -verbose DynamicDispatch.class
    sayHello
{   Last modified 2016-5-10; size 618 bytes
    MD5 checksum 75f543bb48af12eb640e39c05ce80c07
    Compiled from "DynamicDispatch.java"
public class org.fenixsoft.polymorphic.DynamicDispatch
    minor version: 0
    major version: 52
    flags: ACC_PUBLIC, ACC_SUPER
Constant pool:
    #1 = Methodref          #8.#22      // java/lang/Object."<init>":()V
    #2 = Class               #23         // org/fenixsoft/polymorphic/DynamicDispatch$Man
    #3 = Methodref          #2.#22      // org/fenixsoft/polymorphic/DynamicDispatch$Man."<init>":()V
    #4 = Class               #24         // org/fenixsoft/polymorphic/DynamicDispatch$Woman
    #5 = Methodref          #4.#22      // org/fenixsoft/polymorphic/DynamicDispatch$Woman."<init>":()V
    #6 = Methodref          #12.#25     // org/fenixsoft/polymorphic/DynamicDispatch$Human.sayHello:()V
    #7 = Class               #26         // org/fenixsoft/polymorphic/DynamicDispatch
    #8 = Class               #27         // java/lang/Object
    #9 = Utf8                Woman
   #10 = Utf8                InnerClasses
   #11 = Utf8                Man
   #12 = Class               #28         // org/fenixsoft/polymorphic/DynamicDispatch$Human
   #13 = Utf8                Human
   #14 = Utf8                <init>
   #15 = Utf8                ()V
   #16 = Utf8                Code
   #17 = Utf8                LineNumberTable
   #18 = Utf8                main
   #19 = Utf8                ([Ljava/lang/String;)V
   #20 = Utf8                SourceFile
   #21 = Utf8                DynamicDispatch.java
   #22 = NameAndType         #14:#15     // "<init>":()V
   #23 = Utf8                org/fenixsoft/polymorphic/DynamicDispatch$Man
   #24 = Utf8                org/fenixsoft/polymorphic/DynamicDispatch$Woman
   #25 = NameAndType         #29:#15     // sayHello:()V
   #26 = Utf8                org/fenixsoft/polymorphic/DynamicDispatch
   #27 = Utf8                java/lang/Object
   #28 = Utf8                org/fenixsoft/polymorphic/DynamicDispatch$Human
   #29 = Utf8

```

```

public org.fenixsoft.polymorphic.DynamicDispatch();
  descriptor: ()V
  flags: ACC_PUBLIC
  Code:
    stack=1, locals=1, args_size=1
      0: aload_0
      1: invokespecial #1                  // Method java/lang/Object."<init>":()V
      4: return
   LineNumberTable:
      line 7: 0

public static void main(java.lang.String[]);
  descriptor: ([Ljava/lang/String;)V
  flags: ACC_PUBLIC, ACC_STATIC
  Code:
    stack=2, locals=3, args_size=1
      0: new                #2              // class org/fenixsoft/polymorphic/DynamicDispatch$Man
      3: dup
      4: invokespecial #3              // Method org/fenixsoft/polymorphic/DynamicDispatch$Man."<init>":()V
      7: astore_1
      8: new                #4              // class org/fenixsoft/polymorphic/DynamicDispatch$Woman
     11: dup
     12: invokespecial #5              // Method org/fenixsoft/polymorphic/DynamicDispatch$Woman."<init>":()V
     15: astore_2
     16: aload_1
     17: invokevirtual #6              // Method org/fenixsoft/polymorphic/DynamicDispatch$Human.sayHello:()V
     20: aload_2
     21: invokevirtual #6              // Method org/fenixsoft/polymorphic/DynamicDispatch$Human.sayHello:()V
     24: new                #4              // class org/fenixsoft/polymorphic/DynamicDispatch$Woman
     27: dup
     28: invokespecial #5              // Method org/fenixsoft/polymorphic/DynamicDispatch$Woman."<init>":()V
     31: astore_1
     32: aload_1
     33: invokevirtual #6              // Method org/fenixsoft/polymorphic/DynamicDispatch$Human.sayHello:()V
     36: return
   LineNumberTable:
      line 28: 0
      line 29: 8
      line 30: 16
      line 31: 20
      line 32: 24
      line 33: 32
      line 34: 36
  }
SourceFile: "DynamicDispatch.java"
InnerClasses:
  static #9= #4 of #7; //Woman=class org/fenixsoft/polymorphic/DynamicDispatch$Woman of class org/fenixsoft/polymorphic/DynamicDispatch
  static #11= #2 of #7; //Man=class org/fenixsoft/polymorphic/DynamicDispatch$Man of class org/fenixsoft/polymorphic/DynamicDispatch
  static abstract #13= #12 of #7; //Human=class org/fenixsoft/polymorphic/DynamicDispatch$Human of class org/fenixsoft/polymorphic/DynamicDispatch

```

JVM（多态的本质）

多态的实现过程，本质就是方法调用动态绑定的过程。JVM 通过当前栈帧的信息，定位出对象真实类型，找到被调用方法的具体实现，最后使用该方法的直接引用完成方法调用。invokevirtual指令在运行时解析大致分为以下几个步骤：

- Step 1：先从操作栈中找到对象的实际类型C；
- Step 2：找到C中与被调用方法签名相同的方法，如果有访问权限就返回这个方法的直接引用，如果没有访问权限就报错java.lang.IllegalAccessError；
- Step 3：如果第2步找不到相符的方法，就去搜索C的父类，按照继承关系自下而上依次执行第2步的操作；
- Step 4：如果第3步找不到相符的方法，就报错java.lang.AbstractMethodError；

可以看到，如果子类覆盖了父类的方法，则在多态调用中，动态绑定过程会首先确定实际类型是子类，从而先搜索到子类中的方法；

继承中的成员变量

继承中的成员变量

```
class A{  
    public String flag = "Father";  
    public void showFlag(){  
        System.out.println(flag);  
    }  
}
```

```
class B extends A{  
    public String flag = "Son";  
}
```

```
new B().showFlag();
```

```
class A{  
    public String flag = "Father";  
    public void showFlag(){  
        System.out.println(flag);  
    }  
}
```

```
class B extends A{  
    public String flag = "Son";  
    public void showFlag(){  
        System.out.println(flag);  
    }  
}  
new B().showFlag();
```

继承中的成员变量

```
class A2{  
    public String flag = "Father";  
    public void showFlag(){  
        System.out.println(flag);  
    }  
}
```

```
class B2 extends A2{  
    public String flag = "Son";  
    public void showFlag(){  
        System.out.println(flag);  
    }  
}
```

```
A2 a = new B2();  
System.out.println(a.flag);
```

```
A2 a2 = new A2();  
System.out.println(a2.flag);
```

```
B2 b = new B2();  
System.out.println(b.flag);
```

成员变量没有覆盖

- 成员变量能不能访问是跟随引用变量的类型
- 实际调用也是引用变量的类型
 - 因为两个flag根本就是不同的变量
- 变量没有覆盖的说法

java中的变量都是静态绑定的，方法只有static和final(所有private默认是final的,子类不可能修改父类的私有方法)是静态绑定的。编译时已经确切知道程序所要执行的代码，不存在重载等问题。

```
class A2{  
    public String flag = "Father";  
    public void showFlag(){  
        System.out.println(flag);  
    }  
}
```

```
class B2 extends A2{  
    public String flag = "Son";  
    protected void showFlag(){  
        System.out.println(flag);  
    }  
}
```

```
A2 a = new B2();  
System.out.println(a.flag);
```

```
A2 a2 = new A2();  
System.out.println(a2.flag);
```

```
B2 b = new B2();  
System.out.println(b.flag);
```

```
class A2{  
    public String flag = "Father";  
    public void showFlag(){  
        System.out.println(flag);  
    }  
}
```

```
interface Flag{  
    public void showFlag();  
}
```

```
class B2 extends A2 implements Flag{  
    public String flag = "Son";  
    public void showFlag(){  
        System.out.println(flag);  
    }  
}
```

```
new B2().showFlag();  
Flag f = new B2();  
f.showFlag();
```

```
class A {  
    public int i = 1;  
    public static int j = 11;  
    public static String m1() { return "A's static m1"; }  
    public String m2() { return "A's instance m2"; }  
    public String m3() { return "A's instance m3"; }  
}
```

```
class B extends A {  
    public int i = 2;  
    public static int j = 12;  
    public static String m1() { return "B's static m1"; }  
    public String m2() { return "B's instance m2"; }  
}
```

A x = new B();

```
System.out.println("(1) x.i is " + x.i);  
System.out.println("(2) (B)x.i is " + ((B)x).i);  
System.out.println("(3) x.j is " + x.j);  
System.out.println("(4) ((B)x).j is " + ((B)x).j);
```

```
System.out.println("(5) x.m1() is " + x.m1());  
System.out.println("(6) ((B)x).m1() is " +  
    ((B)x).m1());  
System.out.println("(7) x.m2() is " + x.m2());  
System.out.println("(8) x.m3() is " + x.m3());
```

序号	代码	结果	核心原因
1	x.i	1	实例变量无多态 ，只看引用类型A，取父类A.i
2	((B)x).i	2	强制转型为B，引用类型为B，取子类B.i
3	x.j	11	静态变量无多态 ，只看引用类型A，取父类A.j
4	((B)x).j	12	强制转型为B，引用类型为B，取子类B.j
5	x.m1()	A's static m1	静态方法无多态 ，只看引用类型A，调用父类静态方法
6	((B)x).m1()	B's static m1	强制转型为B，引用类型为B，调用子类静态方法
7	x.m2()	B's instance m2	实例方法有多态 ，看实际对象类型B，调用子类重写的方法
8	x.m3()	A's instance m3	B没有重写m3()，调用父类版本